

Hiwonder TurboPi

Design & Build Report

Adit Phanse

8/18/2026

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Introduction	3
2.1 Background & Motivation	3
2.2 Objectives	3
2.3 Scope	4
3. System Overview	4
4. Design & Methodology	5
4.1 Mechanical Design	5
4.2 Electrical / Hardware Design	6
4.3 Software / Control Design	6
Lane Detection and Steering	6
Obstacle Avoidance	7
Traffic Light Recognition	7
Combining the Three Systems	7
5. Implementation	8
Hardware Assembly	8
Development Setup	8
Building and Testing Each System Independently	8
Integration	9
6. Testing & Results	9
6.1 Test Plan	9
6.2 Results	10
6.3 Analysis	10
7. Challenges & Troubleshooting	11
8. Conclusion & Future Work	12
8.1 Conclusion	12
8.2 Future Improvements	12
9. References	13
10. Appendices	13
Appendix A: Full Source Code	13

1. Executive Summary

This report documents the design, construction, and testing of the Hiwonder TurboPi, an autonomous vehicle capable of staying within lane boundaries, detecting and avoiding obstacles, and responding correctly to traffic light signals. The project was undertaken to gain hands-on experience with computer vision, sensor integration, and real-time control systems in a practical autonomous driving context. Development involved building and tuning a lane-detection and steering pipeline, integrating distance sensors for obstacle avoidance, and implementing color-based traffic light recognition. In testing, the TurboPi reliably maintained lane position on both straight and curved track sections, consistently detected and avoided obstacles placed in its path, and correctly identified and responded to red, yellow, and green traffic lights. These results demonstrate a functioning integration of perception, decision-making, and control on a resource-constrained embedded platform.

2. Introduction

2.1 Background & Motivation

I've always been interested in self-driving cars, but a lot of the information about them makes the technology seem like a black box. You hear things like "the car sees the road" or "the car detects obstacles," but not much about how it actually works. I wanted to understand the basics behind it: how a car detects lanes, recognizes obstacles, and knows what to do when it sees something like a red traffic light. The TurboPi was a great way to explore this because it's small and affordable, but still uses a real camera and sensors. The problems I worked on, like lane detection, obstacle avoidance, and traffic light recognition, are similar to the challenges that real autonomous vehicles face, just on a much smaller scale. Building the robot myself also helped me understand the parts that aren't always obvious in theory, like how lighting can affect what the camera sees or how the car should react when multiple things happen at the same time.

2.2 Objectives

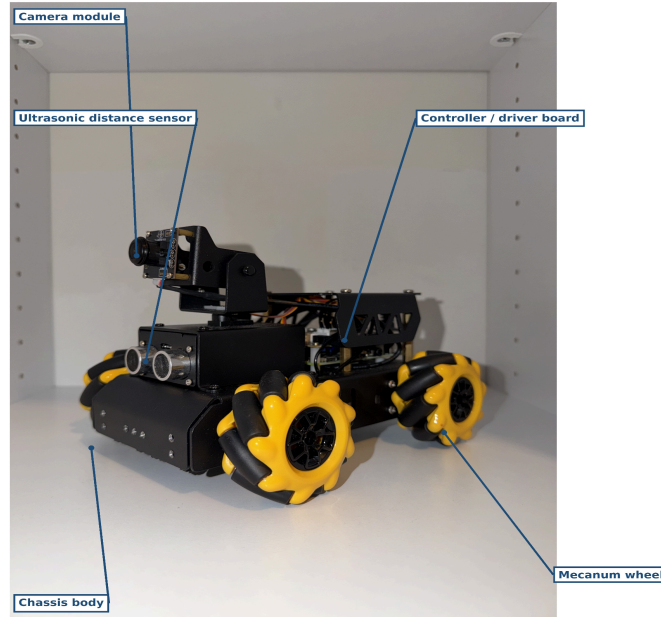
- Build a car that can detect lane lines and steer itself to stay within the lane on both straight and curved sections of the track.
- Detect obstacles in its path and either stop or avoid them without human input.
- Recognize red, yellow, and green traffic lights and respond appropriately to each color.
- Combine lane following, obstacle detection, and traffic light recognition into one control system that can run reliably in real time on the TurboPi.

2.3 Scope

This project focuses on three main autonomous driving behaviors using the Hiwonder TurboPi platform: lane detection and steering, obstacle detection and avoidance, and traffic light recognition with the appropriate response. All development and testing was done on a small test track using painted or taped lane lines, physical obstacles, and a controlled traffic light setup, such as a physical model or printed signal. The project was not tested on real roads and does not attempt to cover more advanced self-driving features like pedestrian detection, lane changes, overtaking, interaction with moving vehicles, GPS navigation, or mapping. The system was also tested under controlled conditions rather than different weather, lighting, or road surfaces. Since the car uses a fixed camera position and pre-tuned detection settings, it may need to be recalibrated if the track layout or lighting changes significantly. Overall, this project should be viewed as a foundation rather than a finished self-driving system. The current build can handle lane following, obstacle avoidance, and traffic light responses, but creating a fully autonomous vehicle capable of operating safely in the real world would require much more hardware, testing, and development. This project is a small-scale step toward understanding and building those systems.

3. System Overview

At a high level, the TurboPi works by continuously capturing frames from its onboard camera and feeding them into three separate processing pipelines: one for lane detection, one for obstacle detection, and one for traffic light recognition. Each pipeline analyzes the current frame (or sensor reading) and outputs a decision, such as "steer left," "stop," or "proceed," and those decisions are combined into a single driving command that gets sent to the motors. The car runs this loop continuously while driving. Under normal conditions, lane detection handles steering. When an obstacle is detected within a set distance, that decision takes priority and the car stops. When a traffic light is in view, its color determines whether the car should continue driving, slow down, or come to a complete stop. This means the system isn't just running three separate features, it's constantly deciding which behavior should take control at any given moment.



4. Design & Methodology

4.1 Mechanical Design

The TurboPi uses a mecanum wheel chassis, so each of its four wheels has small rollers angled around the outside. This allows the car to move sideways and even rotate in place, instead of only moving forward and backward like a regular car. I didn't really use the sideways movement for this project since the main focus was on lane following, obstacle avoidance, and traffic light detection. However, the mecanum wheels still helped make the steering corrections smoother and more precise when the car needed to adjust its position in the lane. The camera is mounted on a raised bracket at the front of the car and angled slightly downward. I set it up this way so it could clearly see the lane lines ahead without picking up too much of the surrounding area. Getting the camera angle right took a bit of trial and error. When it was pointed too far down, the car couldn't see far enough ahead to prepare for upcoming turns. When it was pointed too far up, it captured too much of the background, which made detecting the lane lines more difficult. The ultrasonic sensor is mounted near the front of the car and fairly close to the ground. This was useful because most of the obstacles I used during testing, such as small boxes and blocks, were relatively short. If the sensor was mounted too high, there was a chance it would pass over some of the smaller obstacles without detecting them. The rest of the robot uses the standard TurboPi chassis. The Raspberry Pi, motor driver board, and battery are mounted underneath the top plate, while the camera and ultrasonic sensor are positioned toward the front where they can be used to detect what's happening in front of the car.

4.2 Electrical / Hardware Design

List the electronics: microcontroller, sensors, motors, power supply. Explain how components connect and why they were chosen.

Component	Specification / Model
Main controller	Raspberry Pi 4B
Secondary controller	Onboard STM32-based expansion board (handles low-level motor control and sensor reads, communicates with the Pi over serial)
Motors	4x TT DC gear motors (one per mecanum wheel)
Sensors	Camera module (lane detection, traffic light color); HC-SR04 ultrasonic sensor (obstacle detection); Infrared line sensor array (present on hardware, not used in this project)
Power	7.4V rechargeable Li-ion battery pack

4.3 Software / Control Design

The TurboPi software runs in a continuous loop on the Raspberry Pi. It takes a picture from the camera, processes the image using the different detection systems, decides what the car should do, and then sends commands to the motors. This happens several times per second, allowing the car to react while it is moving. I developed the software directly on the Raspberry Pi using Python and OpenCV for the camera-based image processing. I used RealVNC to access the Raspberry Pi remotely, which made it easier to write and test the code directly on the robot. Instead of developing everything on another computer and transferring the files over, I could make changes and immediately test them using the live camera feed and sensors.

Lane Detection and Steering

The lane detection system processes each camera frame to find the lane lines and figure out whether the car is staying in the center. The basic process is:

1. Convert the image to a color space such as HSV. This makes it easier to separate the lane lines from the background and deal with changes in brightness.
2. Use a color mask or edge detection, such as Canny edge detection, to highlight the lane lines.
3. Focus on the lower part of the camera image since this is the area closest to the car and is most useful for making steering decisions.
4. Estimate where the left and right lane lines are and use them to find the center of the lane.

5. Compare the center of the lane with the center of the camera image to see how far the car has moved to either side.

The difference between these two points is then used to adjust the steering. If the car is far from the center, it makes a stronger correction. If it is only slightly off, the correction is smaller. I used proportional control for these adjustments. There were also situations where the camera could temporarily lose the lane lines, such as during sharp turns or when there was glare. In those cases, the car can briefly use its previous steering direction or slow down while trying to find the lane again.

Obstacle Avoidance

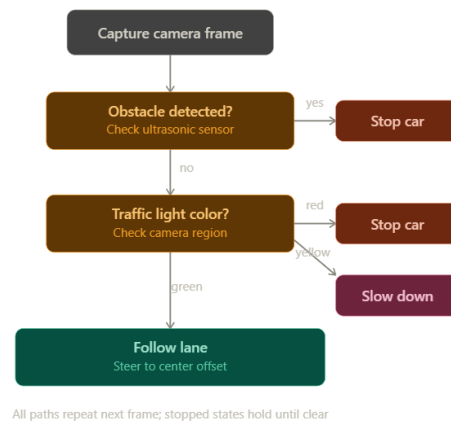
The ultrasonic sensor constantly checks the distance between the car and anything directly in front of it. If an obstacle gets closer than the set distance, the car stops. It stays stopped until the obstacle is no longer in its path. Once the path is clear, the car continues with normal lane following.

Traffic Light Recognition

The traffic light system uses the camera and OpenCV to look for red, yellow, and green areas in the part of the image where the traffic light is expected to be. HSV color masking makes it easier to identify these colors. Once a color is detected, the car responds accordingly. A red light makes the car stop, yellow tells it to slow down or prepare to stop, and green allows it to continue moving.

Combining the Three Systems

Since all three systems are running while the car is driving, there needs to be a clear order for which behavior takes control when multiple things are detected at once. For this project, obstacle detection has the highest priority. Under normal conditions, the lane detection system controls the steering and keeps the car centered. If the ultrasonic sensor detects an obstacle within the set distance, however, the car immediately stops and stays stopped until the obstacle is gone. Traffic lights also take priority over normal lane following. If the camera detects a red light, the car stops and waits for the light to change before continuing. This makes the overall system easier to control and more predictable. Instead of trying to handle several situations at the same time, the car stops whenever it detects something that requires immediate attention. Once the situation is clear, it goes back to normal lane following.



5. Implementation

This section explains how the TurboPi was put together and how the different parts of the software were developed and tested. I started with the hardware setup and then worked on each software system separately before combining everything into one program.

Hardware Assembly

The chassis, mecanum wheels, motors, camera mount, and ultrasonic sensor were assembled using the standard TurboPi setup. I positioned the camera at the angle described in Section 4.1 so that it had a clear view of the lane lines in front of the car. The Raspberry Pi and expansion board were mounted inside the chassis, and the wiring was arranged to keep the motor cables away from the ultrasonic sensor as much as possible to prevent interference with its readings.

Development Setup

All of the code was written and tested directly on the Raspberry Pi using Python. I accessed the Pi remotely through RealVNC, which allowed me to work on the robot from my computer without constantly moving files back and forth. RealVNC was also useful for testing because I could see the camera feed and other output while the program was running. For example, I could display the detected lane lines on the camera feed and see whether the system was actually identifying them correctly.

Building and Testing Each System Independently

I developed the three main systems—lane detection, obstacle avoidance, and traffic light recognition—separately before combining them. This made troubleshooting much easier because I could focus on one problem at a time instead of trying to figure out what was wrong with the entire system.

- **Lane detection:** This was probably the most time-consuming part to get working reliably. Getting the camera feed into OpenCV was fairly straightforward, but finding the right color and edge detection settings took a lot of testing. Even small changes in lighting or the camera angle could affect how well the lane lines were detected.
- **Obstacle avoidance:** I first tested the ultrasonic sensor by itself to make sure the distance readings were consistent. After that, I tested different distances and adjusted the stopping threshold until the car responded reliably when an obstacle was placed in front of it.
- **Traffic light recognition:** This system also used HSV color masking to identify red, yellow, and green. I had to adjust the detection settings so that the system could recognize the traffic light colors without confusing them with other objects in the camera's view.

One thing I noticed throughout the development process was that a setting that worked well under one set of conditions did not always work under another. Changes in lighting, camera position, or the appearance of the track could affect the results. Because of this, getting each system to work reliably required a lot of small adjustments and repeated testing rather than just writing the code once and having everything work immediately.

Integration

Once each system was working reliably on its own, I combined them into a single control loop using the priority system described in Section 4.3. Obstacle and traffic light detection are checked first, while lane following controls the car when neither of those conditions requires attention. Combining the systems introduced a few problems that I had not seen when testing them separately. For example, multiple systems could try to influence the car's behavior at the same time. This meant I had to make sure the priority rules were being followed correctly so that one system did not interfere with another. After making these adjustments, the three systems could work together as a single autonomous driving system.

6. Testing & Results

6.1 Test Plan

The car was tested on a physical track built with taped/painted lane lines, with a physical obstacle placed in its path to test obstacle avoidance. Traffic light recognition was tested separately by holding printed or displayed red, yellow, and green images in front of the car's camera and observing its response. Testing was done iteratively rather than as a single formal trial: the car was run through the track repeatedly over the course of development, with adjustments made to detection thresholds, camera angle, and lighting handling between runs based on how it performed. It took at least 6 trial runs before I was able to properly diagnose and fix the core issues causing inconsistent behavior, and more than ten runs were carried out in total across the full testing process.

6.2 Results

Early test runs frequently failed or behaved inconsistently. Lane detection lost track of the lines under certain lighting, the obstacle avoidance threshold was not yet well-tuned, and traffic light color recognition was inconsistent depending on lighting and camera angle. Nearly every part of the system (lighting sensitivity, threshold values, camera positioning, and reaction timing) required adjustment at some point during testing. It took about 6 trials before the underlying causes of these issues were clearly identified and addressed. Through repeated testing and tuning after that point, performance improved steadily. By the later test runs, the car reliably stayed within the lane, consistently stopped for the obstacle and resumed once it was clear, and correctly identified and responded to all three traffic light colors.

Test Phase	Behavior Observed
Runs 1–6	Frequent lane drift, inconsistent obstacle detection, unreliable traffic light color recognition; root causes being diagnosed
Mid testing	Improved lane tracking and obstacle response after threshold tuning; traffic light recognition still inconsistent under some lighting
Later runs	Reliable lane keeping, consistent obstacle stop-and-resume behavior, accurate traffic light response across all three colors

6.3 Analysis

The final version of the car met all three original objectives: it stayed within the lane, avoided the obstacle, and responded correctly to traffic light colors. The path to that result was not immediate, however. Nearly every component of the system was sensitive to real-world conditions in ways that weren't obvious until testing exposed them: lighting affected both lane and traffic light detection, camera angle affected what the lane detection algorithm could reliably see, and the various thresholds throughout the system all needed to be tuned specifically to the test environment rather than working correctly on default or assumed values. This suggests a broader limitation worth noting: the current system is well-tuned to its specific test track and lighting conditions, but that tuning may not transfer directly to a different environment without repeating a similar tuning process. This is a realistic reflection of a common challenge in real autonomous vehicle systems, where perception systems are often sensitive to the specific conditions they were trained or tuned under.

7. Challenges & Troubleshooting

Lane detection instability

Early on, lane detection was unreliable in several different ways at once: lane lines would sometimes disappear from detection entirely, the car would swerve or overcorrect even when it was roughly centered, and it would occasionally lose the lane completely on curves. This turned out to come from a combination of factors rather than one single bug: the color/edge detection thresholds were too tightly tuned to one lighting condition, and the steering correction was reacting too aggressively to small offset changes. Fixing this took adjusting the detection thresholds to be more tolerant of lighting variation and smoothing out the steering response so small offsets didn't cause sharp overcorrections.

Recovering lane tracking after obstacle avoidance

Obstacle detection itself worked correctly from early on. The distance sensor reliably picked up the obstacle and the car stopped as expected. The real issue was what happened afterward: once the obstacle was removed and the car needed to resume driving, it struggled to re-acquire the lane lines and return to normal lane following. Because the car had been sitting stationary, the camera's view and detection state weren't in the same condition as during continuous driving, which caused the handoff back to lane following to behave inconsistently. This was resolved by making sure the lane detection pipeline re-initialized cleanly when control passed back to it, rather than assuming it could resume exactly where it left off.

Traffic light color confusion

Traffic light recognition initially had trouble distinguishing the light colors from the background behind them. Since the HSV color masking was looking for ranges of red, yellow, and green, similarly colored objects or lighting in the background could get picked up as a false match, or the actual light could blend into a background of a similar tone. This was improved by narrowing the color detection ranges and restricting detection to a smaller, more specific region of the frame where the traffic light was expected to appear, rather than scanning the whole image.

General takeaway

Across all three issues, the common thread was that each pipeline needed to be tuned to more than just "does it work in one test," since conditions like lighting, background, and system state (stationary vs. moving) all affected reliability in ways that weren't obvious until they were tested directly. This is part of why the testing process in Section 6 took multiple iterations rather than succeeding on the first attempt.

8. Conclusion & Future Work

8.1 Conclusion

This project set out to build an autonomous car capable of three core behaviors: staying within lane boundaries, avoiding obstacles, and responding correctly to traffic light signals. All three objectives were ultimately achieved. The TurboPi reliably tracks and stays centered within its lane, detects obstacles in its path and stops until they are cleared, and correctly identifies and responds to red, yellow, and green traffic lights. Getting there was not immediate. Each of the three systems had real issues during development, from lane detection instability, to the car failing to properly resume lane tracking after stopping for an obstacle, to traffic light colors being confused with the background. Diagnosing and fixing these issues took repeated testing and iteration rather than a single clean implementation, which mirrors a real challenge in autonomous vehicle development: systems that work in one condition often fail in another, and reliability comes from testing against real-world variation rather than a single successful run. More broadly, this work demonstrated that the core building blocks of self-driving technology (perception, decision-making, and control) can be understood and implemented hands-on, even at a small scale, and that the challenges involved are genuinely representative of the problems full-scale autonomous vehicles have to solve. What's built so far should be seen as a working foundation rather than a finished result. True full autonomy would require more resources than were available here, including more advanced sensors, greater computing power, and a much wider range of testing conditions. Development is expected to continue as those resources become available.

8.2 Future Improvements

Continued development toward full autonomy, as more resources become available, is the overall direction for this project. The improvements below represent specific next steps in that direction rather than isolated fixes.

- **More robust lighting handling** — the current detection thresholds are tuned to the specific test environment and lighting conditions used during development. Making detection more adaptive to different lighting, rather than relying on fixed threshold values, would make the system more reliable outside of this specific setup.
- **Smarter obstacle handling** — currently the car stops completely and waits for the obstacle to clear. A more advanced version could attempt to steer around static obstacles when there's room, rather than always coming to a full stop.
- **Better recovery after stopping** — the lane-tracking handoff after stopping for an obstacle was one of the trickiest issues to solve during testing. Further refining how the system reinitializes lane detection after any full stop would make this transition more consistent.
- **Additional sensors** — incorporating more advanced sensing, such as a second camera for stereo depth or a LiDAR module, would improve obstacle detection accuracy and reduce reliance on a single ultrasonic sensor.
- **Handling multiple traffic lights or intersections** — the current implementation responds to a single traffic light in view. A more complete system could handle intersections, multiple visible signals, or right-of-way logic.

- **More rigorous quantitative testing** — future testing could track specific numeric metrics, like reaction time in milliseconds or exact stopping distance in centimeters, across many trials, rather than relying on qualitative pass/fail observations.

9. References

Hiwonder, "TurboPi User Manual," Hiwonder Technology.

Hiwonder, "TurboPi Tutorials," YouTube playlist. Available:

<https://www.youtube.com/watch?v=JQAFNUT41JE&list=PLFbzd0m6AcmLzo53o2Tsa20BS350rWGMj>

10. Appendices

Appendix A: Full Source Code

<https://github.com/aditphanse/Hiwonder-TurboPi->